ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		С.А. Рогачев
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Машина Тьюринга

по курсу: ТЕОРИЯ ВЫЧИСЛИТЕЛЬНЫХ ПРОЦЕССОВ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

Цель работы: Целью данной работы является изучение принципов работы машины Тьюринга (МТ) через создание программы на языке высокого уровня, которая имитирует её функционирование. Задача включает разработку программы, способной считывать входные данные из файлов, выполнять заданные переходы и выводить результаты в соответствии с требованиями, а также реализацию контроля возможных ошибок.

Основные сведения из теории:

Машина Тьюринга — это абстрактная вычислительная модель, предложенная Аланом Тьюрингом в 1936 году для исследования алгоритмов и вычислимости. Она состоит из бесконечной ленты, разделённой на ячейки, каждой из которых соответствует символ из конечного алфавита, головки для чтения и записи символов, а также конечного множества состояний и переходных правил. Переходы определяют, какой символ записать, в какое состояние перейти и в каком направлении двигаться (вправо или влево) в зависимости от текущего состояния и читаемого символа. МТ останавливается, когда не находится подходящего перехода или выполняется команда остановки (например, с символом !). Эта модель лежит в основе современной теории вычислений и демонстрирует фундаментальные пределы алгоритмической вычислимости.

Постановка задачи:

Необходимо написать программу для машины Тьюринга (МТ), реализующую вычисление арифметической функции согласно выданному варианту задания, а именно функции $3x-y$ (вариант 39). Для выполнения задания следует использовать приложение Algo2000 для составления совокупности команд P. Аргументы функции задаются набором символов "1", где, например, $2\cdot3$ будет представлено как $11*111$. Работа машины Тьюринга должна начинаться со стандартной начальной конфигурации (состояние q0 и

начальная позиция головки на первом символе ленты) и заканчиваться стандартной конечной конфигурацией (остановка при выполнении команды с индикатором завершения).

В рамках второй части лабораторной работы требуется создать программу на языке высокого уровня, имитирующую работу машины Тьюринга, с соблюдением следующих требований:

- Входная лента машины Тьюринга должна считываться из файла.
- Программа для машины Тьюринга должна считываться из файла.
- Алфавит должен считываться из файла.
- Результат работы программы должен выводиться в файл.
- Результат должен содержать следующие элементы:
    - Состояние ленты перед выполнением каждой команды.
    - Указание положения головки на ленте (символ ^).
    - Выполненную команду в формате, соответствующем примеру: 11∗111= — состояние ленты перед выполнением команды; ^ — положение головки на ленте; q01→q1> — выполняемая команда; 1∗111= — состояние ленты после выполнения команды; ^ — положение головки на ленте.
- В программе должен быть реализован контроль возможных ошибок машины Тьюринга (например, не задан переход, отсутствует символ в алфавите и др.).

Требования к входным данным:

- **Совокупность команд**: Формат записи команд определяется согласно форме adq, где:
    - a — символ, который будет записан в обозреваемую ячейку;
    - d — направление сдвига (< для влево, > для вправо);
    - q — состояние, в которое перейдёт МТ. Каждая команда записывается на отдельной строке в файле.
- **Алфавит**: Символы внешнего алфавита перечислены в файле через пробел.

Для реализации функции 3x−y необходимо разработать соответствующую совокупность команд, учитывая, что x и y задаются как последовательности "1", разделённые символом ∗, а результат должен быть представлен как соответствующее количество "1" после вычисления.

Совокупность команд для машины Тьюринга:

q0 1 -> q1 _ >

q0 _ -> q0 _ >

q1 1 -> q1 1 >

q1 * -> q2 * >

q2 1 -> q3 a >

q3 1 -> q3 1 >

q3 = -> q3 = >

q3 - -> q8 - >

q3 _ -> q4 1 <

q4 1 -> q4 1 <

q4 a -> q2 a >

q4 = -> q9 = <

q5 1 -> q4 1 <

q5 a -> q5 1 <

q5 * -> q6 * <

q6 1 -> q7 1 <

q6 _ -> q10 _ >

q7 1 -> q7 1 <

q7 _ -> q0 _ >

q8 1 -> q8 1 >

q8 = -> q3 = >

q9 1 -> q9 1 <

q9 - -> q5 - <

q10 1 -> q10 _ >

q10 * -> q10 _ >

q10 - -> q11 - >

q11 1 -> q12 a >

q11 a -> q11 a >

q11 = -> q15 _ <

q12 1 -> q12 1 >

q12 = -> q12 = >

q12 _ -> q13 _ <

q13 1 -> q14 _ <

q14 1 -> q14 1 <

q14 a -> q14 a <

q14 = -> q14 = <

q14 - -> q11 - >

q15 1 -> q15 ! >

q15 a -> q15 _ <

q15 - -> q15 _ >

q15 _ -> q15 _ >

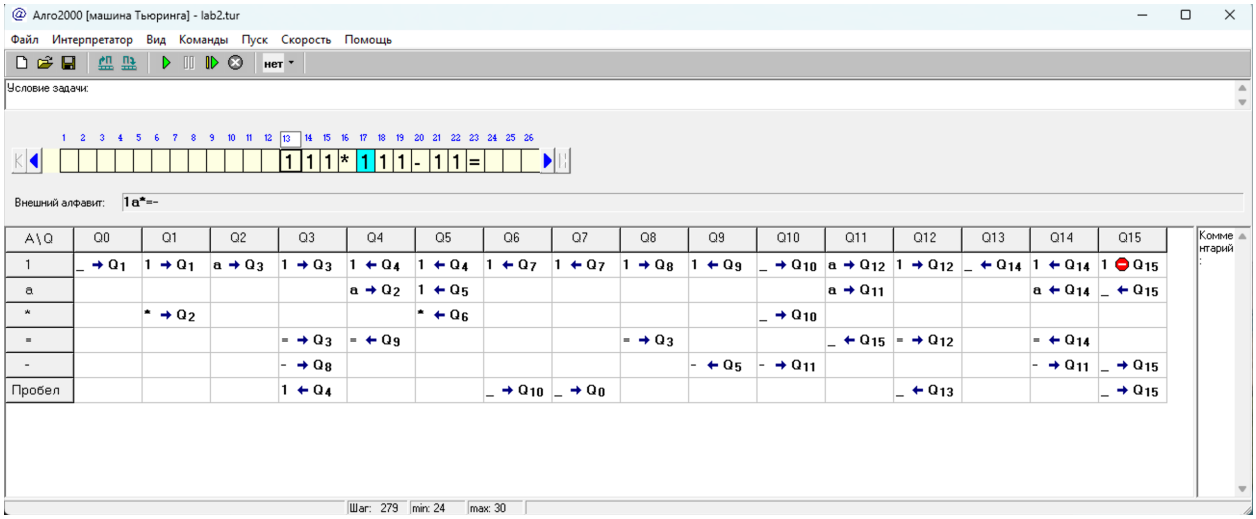
Листинг программы на языке высокого уровня с комментариями:

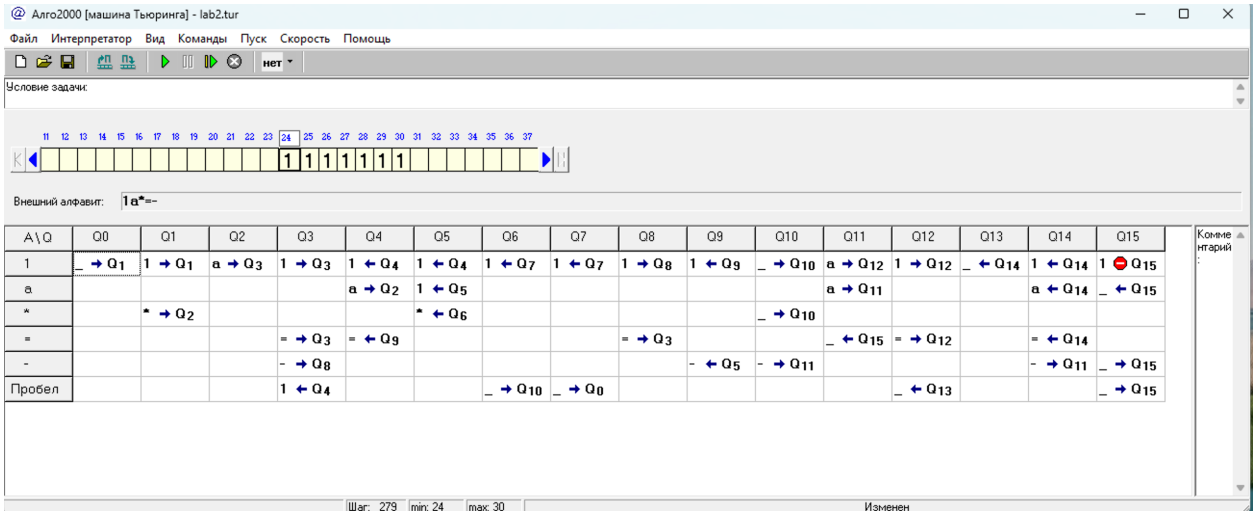
```
import sys

# Функция для загрузки алфавита из файла
def load_alphabet(filename):
    with open(filename, 'r', encoding='utf-8') as f:
        return set(f.read().strip().split())  # Используем set для быстрого поиска

# Функция для загрузки переходов из файла
def load_transitions(filename, alphabet):
    transitions = {}
    with open(filename, 'r', encoding='utf-8') as f:
        for line_num, line in enumerate(f, 1):
            line = line.strip()
            if not line or line.startswith('#'):  # Пропускаем пустые строки или
комментарии
                continue
            parts = line.split()
```

```
            if len(parts) != 6 or parts[2] != '->':
                print(f"Неверный формат в строке {line_num}: {line}", file=sys.stderr)
                continue
            curr_state, curr_sym, _, new_state, new_sym, direction = parts
            if curr_sym not in alphabet:
                print(f"Символ '{curr_sym}' отсутствует в алфавите в строке
{line_num}", file=sys.stderr)
                continue
            if new_sym not in alphabet and new_sym != '!':  # Разрешаем '!' как
специальный символ
                print(f"Новый символ '{new_sym}' отсутствует в алфавите в строке
{line_num}", file=sys.stderr)
                continue
            if direction not in {'>', '<'}:
                print(f"Неверное направление '{direction}' в строке {line_num}",
file=sys.stderr)
                continue
            transitions[(curr_state, curr_sym)] = (new_state, new_sym, direction)
    return transitions

# Функция для загрузки начальной ленты из файла
def load_tape(filename, alphabet, blank='_'):
    with open(filename, 'r', encoding='utf-8') as f:
        tape_str = f.read().strip()  # Предполагаем, что лента — это одна строка без
пробелов
    tape = list(tape_str)
    for sym in tape:
        if sym not in alphabet:
            raise ValueError(f"Неверный символ '{sym}' на начальной ленте, отсутствует
в алфавите")
    return tape

# Симуляция машины Тьюринга
def simulate_tm(transitions, tape, initial_state='q0', blank='_',
output_file='output.txt'):
    current_state = initial_state
    head = 0  # Начало с позиции 0
    with open(output_file, 'w', encoding='utf-8') as out:
        while True:
            # Расширение ленты, если головка выходит за пределы
            while head < 0:
                tape.insert(0, blank)
                head += 1
            while head >= len(tape):
                tape.append(blank)

            curr_sym = tape[head]

            # Вывод текущего состояния ленты и позиции головки
            tape_out = ''.join(tape)
            out.write(tape_out + '\n')
            out.write(' ' * head + '^' + '\n')
```

```
            # Проверка перехода
            key = (current_state, curr_sym)
            if key not in transitions:
                out.write(f"Нет перехода для состояния {current_state} и символа
{curr_sym}, остановка.\n")
                break

            new_state, new_sym, direction = transitions[key]

            # Вывод выполненной команды с пробелами
            dir_char = '>' if direction == '>' else '<'
            cmd = f"{current_state} {curr_sym} → {new_state} {new_sym} {dir_char}"
            out.write(cmd + '\n')

            # Применение перехода
            if new_sym == '!':
                out.write("Машина Тьюринга остановлена по команде '!'.\n")
                break
            tape[head] = new_sym
            if direction == '>':
                head += 1
            else:
                head -= 1
            current_state = new_state

# Главный блок выполнения
if __name__ == "__main__":
    try:
        alphabet = load_alphabet('alphabet.txt')
        transitions = load_transitions('program.txt', alphabet)
        tape = load_tape('tape.txt', alphabet)
        simulate_tm(transitions, tape)
        print("Симуляция завершена. Результат записан в output.txt")
    except Exception as e:
        print(f"Ошибка: {e}", file=sys.stderr)
```

Пример результата выполнения:





output.txt

```
883   ____________________________-aa_1111111_______________
885   q15 a → q15 _ <
886   ____________________________-a__1111111_______________
887   |||||||||||||||||||||||||||^
888   q15 a → q15 _ <
889   ____________________________-___1111111_______________
890   |||||||||||||||||||||||||||^
891   q15 - → q15 _ >
892   ____________________________1111111_______________
893   |||||||||||||||||||||||||||^
894   q15 _ → q15 _ >
895   ____________________________1111111_______________
896   |||||||||||||||||||||||||||^
897   q15 _ → q15 _ >
898   ____________________________1111111_______________
899   |||||||||||||||||||||||||||^
900   q15 _ → q15 _ >
901   ____________________________1111111_______________
902   |||||||||||||||||||||||||||^
903   q15 1 → q15 ! >
904   Машина Тьюринга остановлена по команде '!'.
905
```

Вывод:

В ходе выполнения лабораторной работы я глубоко изучил принципы работы машины Тьюринга, освоив её теоретические основы и практическое применение для вычисления арифметических функций, таких как 3x - y (вариант 39), где аргументы представлены последовательностями "1", а также получил опыт работы с приложением Algo2000 для составления и тестирования переходов. Я научился разрабатывать программу на языке Python, которая имитирует функционирование МТ, включая считывание данных из файлов (алфавит, команды и ленту), реализацию переходов с учётом направлений и записи символов, а также контроль ошибок, таких как отсутствие переходов или неверные символы. Практическое освоение позволило мне устранить изначальные проблемы, такие как дублирование строк в выводе и слияние символов в командах (например, q31 -> q31 >), добавив пробелы для читаемости, а также внедрить явную остановку при выполнении команды с !, что укрепило мои навыки программирования, понимание вычислительных моделей и уверенное использование инструмента Algo2000 для разработки алгоритмов.